



UNIVERSITY OF PISA
MASTER'S DEGREE IN COMPUTER SCIENCE,
ARTIFICIAL INTELLIGENCE

**Computational Mathematics for Learning &
Data Analysis (646AA)**

Group number: 63

Members:
Matthew Bernardi
Gabriele Marsili

ACADEMIC YEAR: 2025/2026

Contents

1	Introduction to the problem	2
1.1	Problem 25 ML	2
1.2	Extreme Learning Machine	2
1.3	Least-Squares Problem	3
1.4	Regularization Techniques	4
1.4.1	L_1 regularization	4
1.5	Objective Function Analysis	5
1.5.1	Convexity	5
1.5.2	Non-smoothness	5
1.5.3	Optimality conditions	6
2	Algorithm 1: Iteratively Reweighted Least Squares	7
2.1	The core idea: approximating the L_1 norm	7
2.2	The weighted least-squares subproblem	8
2.3	The Majorization-Minimization interpretation	9
2.4	The complete algorithm	10
2.5	Convergence analysis	11
2.6	Computational cost	11
2.7	Expected behavior on our problem	12
3	Algorithm 2: Deflected Subgradient Method	13
3.1	Motivation: limitations of the plain subgradient method	13
3.1.1	Key properties of nonsmooth convex functions	14
3.1.2	The deflection idea	15
3.2	Convergence conditions: balancing deflection and stepsize	15
3.2.1	Optimal deflection parameter	16
3.3	The target-level mechanism	16
3.3.1	The complete algorithm	17
3.3.2	Parameters	18
3.4	Application to our problem	18
3.4.1	Subgradient computation	18
3.4.2	Convergence	19
3.4.3	Expected properties on the ELM problem	20

4	Algorithm Comparison	22
4.1	Core strategy: how each algorithm handles non-smoothness . . .	22
4.2	Convergence behaviour	23
4.2.1	Monotonicity	23
4.2.2	Rate	23
4.2.3	Convergence on the ELM problem	23
4.3	Computational cost	24
4.3.1	Per-iteration cost	24
4.3.2	Total cost	24
4.4	Solution quality	25
4.4.1	Sparsity	25
4.4.2	Accuracy	25
4.5	Parameter sensitivity	25
4.6	Summary and guidelines for the ELM problem	26

Chapter 1

Introduction to the problem

1.1 Problem 25 ML

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg \min_w f(w)$, with $f(w) = \|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg \min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient) Use a threshold so that the values in W do not get too large if $(w_k)_i \approx 0$. No off-the-shelf solvers allowed.

1.2 Extreme Learning Machine

Extreme Learning Machine (ELM) is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W}_1 \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training).
- $\sigma(\cdot)$ is the element-wise activation function applied to the $\mathbf{W}_1 \mathbf{x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector.
- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector, chosen by solving the least-squares problem.

- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W}_1 \mathbf{x}) \in \mathbb{R}$ is the prediction of the model.

The key insight behind this architecture is that randomly fixing $\mathbf{W}_1$ transforms an extremely hard non-linear optimization problem (training a full neural network) into a simpler **linear system** resolution. Classical neural networks must learn *all* weights jointly, which requires non-convex optimization with no guarantee of finding the global minimum. In an ELM, the only free parameters are the output weights $\mathbf{w}$, and once $\mathbf{W}_1$ is fixed, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1)$ are constant. Finding $\mathbf{w}$ then reduces to solving the Least-Squares problem, which is convex with a closed-form solution. This seemingly drastic choice works surprisingly well in practice: random projections through a non-linear activation function are sufficient to produce rich, expressive features from which a linear model can learn effectively, exploiting the same properties of expressiveness as the **linear basis expansions** in classical Machine Learning models (e.g. linear models).

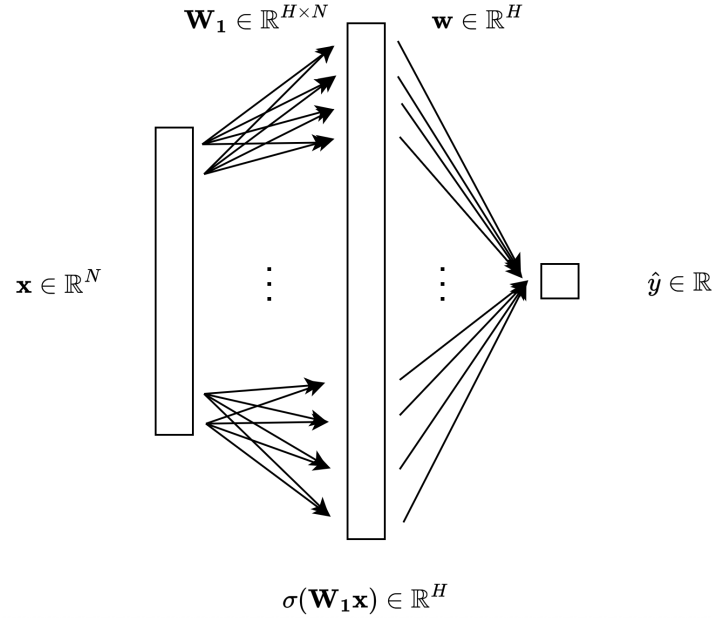


Figure 1.1: Visual representation of an Extreme Learning Machine model.

1.3 Least-Squares Problem

The Least-Squares Problem is an optimization problem used in machine learning for data fitting. It aims at finding the best curve that minimizes the sum of squared differences between the *predicted value* $A\mathbf{x}$ and the observed *ground*

truth (or *target*) $\mathbf{y}$. Mathematically, we can represent this problem as follows:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{y}\|_2^2 \quad (1.1)$$

In our case, a more suited definition is the following one, considering $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1) \in \mathbb{R}^{M \times H}$ the matrix of activations of the hidden layer, while $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, and $\mathbf{y} \in \mathbb{R}^M$ is the vector of all the target values:

$$\min_{\mathbf{w}} \|\mathbf{Xw} - \mathbf{y}\|_2^2 \quad (1.2)$$

The Least-Squares problem is proven to have a unique solution if and only if the matrix A has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{y}^T \mathbf{X} \mathbf{w} + \frac{1}{2} \mathbf{y}^T \mathbf{y} \quad (1.3)$$

since the gradient of this function is $\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}$, the Hessian matrix is $\mathbf{X}^T \mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} = 0$$

or

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

where $\mathbf{X}^T \mathbf{X}$ is square invertible since it's positive definite. The linear system has a unique solution. We can solve this equation using methods such as *IRLS* and *Deflected Subgradient Methods*.

1.4 Regularization Techniques

When solving the Least-Squares Problem, we may prefer solutions that use a subset of available features. This prevents the problem of overfitting the model to the data, and producing a result that is more robust with respect to the data given. Regularization is the technique used to add a penalty to weights, and control the free parameters of the model to avoid overfitting to data.

1.4.1 L_1 regularization

The L_1 regularization, also called Lasso, tends to bring many weights to zero. This is a form of implicit *feature selection* mechanism. The L_1 regularization is the best convex approximation for the L_0 norm, which penalizes the number of non-zero weights. Because of this sparsity, we are going to use the L_1 instead of L_2 , which shrinks the weights but rarely zeroes them out.

When adding the L_1 regularization term to the Least-Squares problem, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \sum_i^H |w_i| \quad (1.4)$$

The price to pay when using the Lasso regularization is the non-differentiability of the loss function $f(\mathbf{w})$ for each point where $w_i = 0$ for at least one i index. This is because of the gradient, which can be computed correctly only in the points where $w_i \neq 0 \ \forall i$, and is the following:

$$\nabla f(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \text{sign}(\mathbf{w})$$

Because of that, we can use the Deflected Subgradient Methods (2) and the IRLS algorithm (2), which we will see in the following chapters.

1.5 Objective Function Analysis

The full objective function we need to minimize is the Lasso regularization of the Least-Squares problem:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda \|\mathbf{w}\|_1}_{h(\mathbf{w})}$$

It is useful to analyse the mathematical properties of $f(\mathbf{w})$, as they determine which algorithms can be applied.

1.5.1 Convexity

$f(\mathbf{w})$ is the sum of two convex functions. The quadratic term $g(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ has Hessian $\nabla^2 g(\mathbf{w}) = 2\mathbf{X}^T\mathbf{X} \succeq 0$, making it convex. If $\mathbf{X}$ has full column rank, then $\mathbf{X}^T\mathbf{X} \succ 0$ and g is *strictly convex*. The regularization term $h(\mathbf{w}) = \lambda \|\mathbf{w}\|_1$ is convex as a scaled norm. Therefore $f = g + h$ is convex; and if $\mathbf{X}$ has full column rank, f is **strictly convex** and admits a **unique global minimum**.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M \times H}$ has full column rank, then $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$.*

1.5.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty $h(\mathbf{w}) = \lambda \|\mathbf{w}\|_1$ is **not differentiable** at any point where some component $w_i = 0$. This means f itself is non-smooth, and we cannot rely on gradient-based methods alone. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \text{sign}(\mathbf{w})$$

where $\text{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 2).

1.5.3 Optimality conditions

Since f is non-smooth, the classical condition $\nabla f(\mathbf{w}^*) = 0$ does not apply at every minimizer. Instead, the optimality condition is expressed via the **subdifferential**: $\mathbf{w}^*$ is a global minimizer if and only if:

$$0 \in \partial f(\mathbf{w}^*) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda \partial \|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise by:

$$[\partial \|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

This condition can be written component-wise as: for each $i = 1, \dots, H$,

$$[2\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})]_i + \lambda s_i = 0, \quad s_i \in \partial |w_i^*|.$$

Intuitively, λ controls the sparsity of $\mathbf{w}^*$: the larger λ is, the easier it is for the optimality condition to force components $w_i^* = 0$, producing a sparser solution.

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p -norm. Its core idea is to handle non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits an efficient closed-form solution. In our setting, IRLS is used to minimize the L_1 -regularized least-squares objective:

$$\min_{\mathbf{w} \in \mathbb{R}^H} f(\mathbf{w}) = \underbrace{\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda\|\mathbf{w}\|_1}_{h(\mathbf{w})} \quad (2.1)$$

where $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, and $\lambda > 0$ is the regularization hyperparameter.

2.1 The core idea: approximating the L_1 norm

The difficulty with problem (2.1) is the L_1 penalty term $h(\mathbf{w}) = \lambda\|\mathbf{w}\|_1 = \lambda \sum_{i=1}^H |w_i|$: it is convex, but has sharp corners (*kinks*) wherever any $w_i = 0$, making it non-differentiable in those points. Classical gradient-based algorithms cannot be applied directly.

The key observation is the following identity: for any $w_{k,i} \neq 0$,

$$|w_i| = \frac{w_i^2}{|w_i|} \approx \frac{w_i^2}{|w_{k,i}|} \quad (2.2)$$

where $w_{k,i}$ is the value of the i -th component at the current iteration k . This is a *quadratic approximation* of $|w_i|$ in the variable w_i , where the denominator $|w_{k,i}|$ is treated as a fixed constant. Summing over all components, we obtain:

$$\|\mathbf{w}\|_1 = \sum_{i=1}^H |w_i| \approx \sum_{i=1}^H \frac{w_i^2}{|w_{k,i}|} = \mathbf{w}^T \mathbf{W}_k \mathbf{w} \quad (2.3)$$

where $\mathbf{W}_k \in \mathbb{R}^{H \times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |w_{k,i}|^{-1/2} \quad (2.4)$$

This turns the non-smooth L_1 term into a *smooth, quadratic* expression $\mathbf{w}^T \mathbf{W}_k \mathbf{w}$. Intuitively, $\mathbf{W}_k$ applies *personalized penalization* to each component of $\mathbf{w}$: components with small magnitude at iteration k receive a large weight (strong penalization, pushing them toward zero), while components with large magnitude receive a small weight (weaker penalization, leaving them relatively free).

Handling near-zero components. Equation (2.4) becomes numerically unstable when $w_{k,i} \approx 0$, since $|w_{k,i}|^{-1}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \max(|w_{k,i}|, \varepsilon_{\text{thr}})^{-1/2} \quad (2.5)$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$. Setting ε_{thr} too large degrades the approximation of $|w_i|$ and may prevent convergence to a truly sparse solution; setting it too small risks numerical instability. This parameter is an important implementation choice.

2.2 The weighted least-squares subproblem

At each iteration k , substituting the quadratic approximation (2.3) into the objective (2.1) gives the following subproblem:

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + 2\lambda \|\mathbf{W}_k \mathbf{w}\|_2^2 \quad (2.6)$$

This is a standard **weighted least-squares problem with L_2 regularization** (Ridge regression), which is smooth and strictly convex. Its closed-form solution follows from setting the gradient to zero.

Proposition 2.1 (Weighted normal equations). *The unique solution of (2.6) satisfies:*

$$(\mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{W}_k^T \mathbf{W}_k) \mathbf{w}_{k+1} = \mathbf{X}^T \mathbf{y} \quad (2.7)$$

Proof. The gradient of the objective in (2.6) is:

$$\nabla f_k(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 4\lambda \mathbf{W}_k^T \mathbf{W}_k \mathbf{w}$$

Setting this to zero and dividing by 2 yields (2.7). The coefficient matrix $\mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{W}_k^T \mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T \mathbf{X} \succeq 0$ and $2\lambda \mathbf{W}_k^T \mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. $\square$

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{W}_k^T \mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T \mathbf{y}$, equation (2.7) reduces to:

$$\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b} \quad (2.8)$$

This is an $H \times H$ symmetric positive definite (SPD) linear system, which can be solved efficiently at each iteration. Note that $\mathbf{b} = \mathbf{X}^T \mathbf{y}$ can be pre-computed once before the iterative loop, since it does not depend on k . Similarly, $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ is pre-computed once, and only the diagonal $2\lambda \mathbf{W}_k^T \mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.
- **Conjugate Gradient (CG):** an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

Optimality condition For a smooth function f , we know that the minimum is where the gradient of f is equal to 0. In the L_1 regularization problem, we know that we cannot use the gradient directly, but we need the **subdifferential**: the set of "almost-gradients". We know that we're in the minimum if 0 is contained in the subdifferential of the function f , so $0 \in \partial f(\mathbf{w}^*)$.

For our problem:

$$\partial f(w) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \partial \|\mathbf{w}\|_1,$$

where the subdifferential of the L_1 regularization is:

$$[\partial \|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

The condition $0 \in \partial f(\mathbf{w}^*)$ can be written component by component: for each $i = 1, \dots, n$,

$$[2\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})]_i + \lambda s_i = 0, \quad s_i \in \partial |w_i^*|. \quad (2.9)$$

2.3 The Majorization-Minimization interpretation

IRLS can be understood within the broader framework of **Majorization-Minimization (MM) algorithms**. At each iteration, rather than minimizing the original objective $f(\mathbf{w})$ directly, IRLS builds a *surrogate function* $Q(\mathbf{w}, \mathbf{w}_k)$ that:

1. **Majorizes** f : $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$;

2. **Touches** f at the current iterate: $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$;
3. Is **easy to minimize**: $Q(\cdot, \mathbf{w}_k)$ is a smooth quadratic.

In our case, the surrogate is:

$$Q(\mathbf{w}, \mathbf{w}_k) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + 2\lambda\mathbf{w}^T\mathbf{W}_k\mathbf{w} \quad (2.10)$$

The MM construction guarantees a key property: since minimizing $Q(\cdot, \mathbf{w}_k)$ gives $\mathbf{w}_{k+1}$, we have $Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, and since $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k)$, it follows that:

$$f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k) \quad (2.11)$$

That is, **IRLS monotonically decreases the objective function** at every iteration. This is a much stronger guarantee than what the Deflected Subgradient Method provides (see Chapter 3).

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS) for LASSO

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, k_{max}

```

1: Pre-compute  $\mathbf{A} \leftarrow \mathbf{X}^T\mathbf{X}$ ,  $\mathbf{b} \leftarrow \mathbf{X}^T\mathbf{y}$ 
2: Initialize  $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$  (ordinary least-squares solution)
3: for  $k = 0, 1, 2, \dots, k_{\text{max}}$  do
4:   Compute weights:  $(\mathbf{W}_k)_{ii} \leftarrow \max(|w_{k,i}|, \varepsilon_{\text{thr}})^{-1/2}$  for all  $i$ 
5:   Assemble system:  $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda\mathbf{W}_k^T\mathbf{W}_k$ 
6:   Solve:  $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$  (via Cholesky or CG)
7:   if  $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| / \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$  then
8:     break (convergence reached)
9:   end if
10: end for
11: return  $\mathbf{w}_{k+1}$ 

```

Initialization. The algorithm is initialized with $\mathbf{w}_0 = \mathbf{A}^{-1}\mathbf{b}$, the solution to the unregularized least-squares problem. This provides a reasonable starting point: the initial weights $\mathbf{W}_0$ reflect the magnitude of the unregularized solution, giving the algorithm sensible information about which components are likely to be large.

Stopping criterion. The algorithm terminates when the relative change between successive iterates falls below $\varepsilon_{\text{stop}}$:

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}$$

The normalization by $\max(1, \|\mathbf{w}_k\|)$ makes the criterion scale-invariant, avoiding a false trigger when $\|\mathbf{w}_k\|$ is very small.

2.5 Convergence analysis

Monotone decrease. As derived in Section 2.3 from the MM framework, IRLS monotonically decreases the objective: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ for all k . Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency). *If the sequence $\{\mathbf{w}_k\}$ generated by IRLS converges to a point $\mathbf{w}^*$ such that $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i , then $\mathbf{w}^*$ satisfies the optimality conditions (2.9) of the original problem (2.1).*

Proof sketch. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, the weighted normal equations (2.7) give:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^* = 0$$

Since $(\mathbf{W}_*)_{ii} = |w_i^*|^{-1}$ and $(\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^*)_i = |w_i^*|^{-1}w_i^* = \text{sign}(w_i^*)$, this reduces to:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda \text{sign}(\mathbf{w}^*) = 0$$

which is exactly the first-order optimality condition $0 \in \partial f(\mathbf{w}^*)$ when all components $w_i^* \neq 0$. $\square$

Convergence rate. IRLS converges typically in a **linear** (geometric) fashion: the error $\|\mathbf{w}_k - \mathbf{w}^*\|$ decreases approximately by a constant factor at each iteration. On a semilogarithmic plot of $f(\mathbf{w}_k) - f^*$ versus iteration number, this manifests as a straight line (linear decrease in log scale).

The convergence rate is governed by the condition number $\kappa(\mathbf{Q}_k) = \kappa(\mathbf{X}^T\mathbf{X} + 2\lambda\mathbf{W}_k^T\mathbf{W}_k)$ of the linear systems being solved. The L_1 regularization term $2\lambda\mathbf{W}_k^T\mathbf{W}_k$ acts as a preconditioner that adds a positive diagonal to $\mathbf{X}^T\mathbf{X}$, which generally *improves* conditioning compared to the unregularized system. In practice, IRLS reaches high accuracy in 10–50 iterations.

Remark 2.3 (Superlinear rate for ℓ_p with $p < 1$). It has been proved that for ℓ_p minimization with $p < 1$, IRLS converges with a *superlinear* (faster than geometric) rate under the restricted isometry property (RIP). For our ℓ_1 case ($p = 1$), a global linear convergence rate has been established under the null space property of the measurement matrix, ensuring that convergence occurs from any initialization.

2.6 Computational cost

The per-iteration cost of IRLS breaks down as follows:

- **Weight update $\mathbf{W}_k$:** $O(H)$ — simply iterating over the components of $\mathbf{w}_k$.

- **Matrix update** $\mathbf{Q}_k = \mathbf{A} + 2\lambda \mathbf{W}_k^T \mathbf{W}_k$: $O(H)$ — since $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve** $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$: $O(H^3/3)$ with Cholesky, or $O(kH^2)$ with CG for k CG steps.

The dominant cost per iteration is the linear system solve: $O(H^3)$ with Cholesky. However, the total number of IRLS iterations is typically small (10–50), and the pre-computation of $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ (costing $O(MH^2)$) is amortized over all iterations. The overall cost is therefore:

$$O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$$

For problems where $H \ll M$ (many training samples, moderate hidden layer size), the pre-computation dominates and IRLS is very efficient.

2.7 Expected behavior on our problem

Based on the theoretical analysis, the following behavior is expected when applying IRLS to the ELM LASSO problem (2.1):

- **Monotone convergence.** The objective $f(\mathbf{w}_k)$ decreases at every iteration, making IRLS straightforward to monitor and debug. There are no oscillations.
- **Linear convergence rate.** On a semilogarithmic plot of $f(\mathbf{w}_k) - f^*$ versus iteration number, the curve appears as an approximately straight line. The slope depends on the conditioning of $\mathbf{Q}_k$ and on ε_{thr} .
- **Sparse solutions.** As iterations proceed, components $w_{k,i}$ that are small receive increasingly large weights, which pushes them further toward zero. At convergence, many components are *exactly zero* (or within ε_{thr} of zero), yielding a genuinely sparse weight vector.
- **Effect of λ .** Larger values of λ produce sparser solutions but higher training error. Smaller λ retains more components but may overfit. The convergence speed may also vary with λ , since it affects the conditioning of $\mathbf{Q}_k$.
- **Sensitivity to ε_{thr} .** A smaller ε_{thr} yields a better approximation to the true L_1 norm and promotes more exact sparsity, but risks numerical instability if components approach zero very closely. A value around 10^{-8} is typically a good compromise.

Chapter 3

Algorithm 2: Deflected Subgradient Method

Notation. In this chapter we present the deflected subgradient method in its general form, using $\mathbf{x}^i$ for the iterates. When applied to our specific LASSO problem (Section 3.4 onward), the iterates are denoted $\mathbf{w}_i$ to match the notation of Chapter 1.

We choose the *stepsize-restricted* variant with *target level* (SGPTL) and *optimal deflection*, because it avoids the need for a priori knowledge of f^* while retaining the practical efficiency of the Polyak stepsize, and the optimal deflection minimises the direction norm, maximising progress per iteration.

3.1 Motivation: limitations of the plain subgradient method

The starting point for this algorithm is the *subgradient method*, the natural extension of the gradient method to convex nondifferentiable functions. Given $\mathbf{g}^i \in \partial f(\mathbf{x}^i)$, the update rule mirrors that of the gradient method:

$$\mathbf{x}^{i+1} = \mathbf{x}^i - \alpha^i \mathbf{g}^i. \quad (3.1)$$

However, this method suffers from two structural limitations. First, it uses only local information (the subgradient at the current point) and retains no memory of previous iterations. The result is a pronounced zig-zag behaviour, where the algorithm oscillates between opposing directions, leading to very slow convergence. Second, the stepsize α^i must be chosen with great care. A *constant* stepsize does not work: the algorithm cycles forever without converging. The stepsize must tend to zero, but not too fast: the standard requirement is the *Diminishing-Square Summable* (DSS) condition

$$\sum_{i=1}^{\infty} \alpha^i = \infty \quad \text{and} \quad \sum_{i=1}^{\infty} (\alpha^i)^2 < \infty, \quad (3.2)$$

satisfied, for example, by $\alpha^i = c/i$. The DSS rule guarantees convergence, but the practical speed is very poor. These issues are analogous to the well-known zig-zag of the steepest-descent method on ill-conditioned smooth functions, which is cured by *conjugate directions* (Heavy Ball, conjugate gradient). In the nonsmooth setting, the corresponding remedy is **deflection**.

Intrinsic difficulty of nonsmooth optimisation. Before introducing the algorithm, it is important to understand the fundamental complexity gap between smooth and nonsmooth convex optimisation:

Regularity	Convexity	Complexity
$f \in C^1$, L -smooth	τ -strongly convex	$O(\log(1/\varepsilon))$
$f \notin C^1$, L -Lipschitz	τ -strongly convex	$\Omega(L^2/\varepsilon)$
$f \in C^1$, L -smooth	convex	$O(1/\sqrt{\varepsilon})$
$f \notin C^1$, L -Lipschitz	convex	$\Omega(L/\varepsilon^2)$

The gap between the smooth strongly convex case ($O(\log(1/\varepsilon))$) and the nonsmooth convex case ($\Omega(1/\varepsilon^2)$) is enormous: to reach $\varepsilon = 10^{-6}$, the former requires ≈ 14 iterations while the latter requires $\approx 10^{12}$.

3.1.1 Key properties of nonsmooth convex functions

The theoretical validity of subgradient methods rests on a few fundamental properties of nonsmooth convex functions that generalise the familiar smooth theory.

- **Subdifferential non-emptiness.** For any convex function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and any point $\mathbf{x}$, the subdifferential $\partial f(\mathbf{x})$ is non-empty. This guarantees that a subgradient can always be computed, making each iteration of the algorithm well-defined.
- **Subgradient inequality.** For every $\mathbf{g} \in \partial f(\mathbf{x})$ and every $\mathbf{y} \in \mathbb{R}^n$,

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \langle \mathbf{g}, \mathbf{y} - \mathbf{x} \rangle.$$

This is the analogue of the first-order condition for smooth convex functions, and is the key inequality used in all convergence proofs: it ensures that a step in direction $-\mathbf{g}$ makes progress towards the minimiser.

- **Lipschitz continuity.** A convex function is *locally Lipschitz* on the interior of its domain: for any compact set K , there exists $L_K > 0$ such that $|f(\mathbf{x}) - f(\mathbf{y})| \leq L_K \|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y} \in K$. Lipschitz continuity implies that subgradients are uniformly bounded on compact sets: $\|\mathbf{g}\| \leq L_K$ for all $\mathbf{g} \in \partial f(\mathbf{x})$, $\mathbf{x} \in K$.
- **No gradient at kinks.** Unlike smooth functions, $\partial f(\mathbf{x})$ is a set (not a singleton) wherever f is non-differentiable. The size of this set grows

larger at sharper kinks. The minimum-norm element $\mathbf{g}^* = \arg \min\{\|\mathbf{g}\| : \mathbf{g} \in \partial f(\mathbf{x})\}$ is the most useful single representative at non-smooth points, as it corresponds to the steepest descent direction.

These properties justify using a subgradient in place of the gradient and underpin the convergence guarantee in Theorem 3.3.

3.1.2 The deflection idea

Deflected subgradient methods replace the raw subgradient $\mathbf{g}^i$ with a *deflected direction* $\mathbf{d}^i$, defined as a convex combination of the current subgradient and the previous direction:

$$\mathbf{d}^i = \gamma^i \mathbf{g}^i + (1 - \gamma^i) \mathbf{d}^{i-1}, \quad \mathbf{d}^0 = \mathbf{g}^0, \quad (3.3)$$

where $\gamma^i \in (0, 1]$ is the **deflection parameter**. The iterate is then updated as

$$\mathbf{x}^{i+1} = \mathbf{x}^i - \alpha^i \mathbf{d}^i. \quad (3.4)$$

This can be seen as a nonsmooth analogue of the conjugate gradient method, where the deflected direction $\mathbf{d}^i$ plays the role of the conjugate direction. The parameter γ^i controls the trade-off between responsiveness and memory:

- $\gamma^i = 1$: only the current subgradient is used (plain subgradient method). Maximum local responsiveness, but oscillatory behaviour.
- γ^i small: the direction is dominated by $\mathbf{d}^{i-1}$, producing a smoother trajectory at the risk of ignoring new information.
- Intermediate γ^i : the best compromise between reactivity and smoothness.

3.2 Convergence conditions: balancing deflection and stepsize

The joint choice of γ^i (deflection) and α^i (stepsize) must be carefully coordinated: if both are too large the algorithm may diverge. Two complementary frameworks guarantee convergence.

Approach 1: stepsize-restricted. The deflection γ^i is chosen freely, and the stepsize is constrained via a Polyak-type rule:

$$\alpha^i = \frac{\beta^i (f(\mathbf{x}^i) - f^*)}{\|\mathbf{d}^i\|^2}, \quad \beta^i \leq \gamma^i. \quad (3.5)$$

Since f^* is unknown in practice, it is replaced by the target level $f_{\text{ref}} - \delta$ (see Section 3.3). The condition $\beta^i \leq \gamma^i$ ensures that the step shrinks as the deflection grows stronger (γ^i small). The rationale is that a large deflection injects the “old” direction $\mathbf{d}^{i-1}$ into $\mathbf{d}^i$, which may no longer be a descent direction; a shorter step compensates for this risk.

Approach 2: deflection-restricted. The stepsize α^i is chosen a priori (e.g. via a DSS rule), and the deflection is constrained from below:

$$\gamma^i \geq \frac{\alpha^{i-1} \|\mathbf{d}^{i-1}\|^2}{(f(\mathbf{x}^i) - f^*) + \alpha^{i-1} \|\mathbf{d}^{i-1}\|^2}. \quad (3.6)$$

Again, f^* is replaced by the target level in the actual implementation. As the algorithm converges ($f(\mathbf{x}^i) \rightarrow f^*$), the term $(f(\mathbf{x}^i) - f^*)$ in the denominator vanishes, so the right-hand side tends to 1, i.e. $\gamma^i \rightarrow 1$: near the solution the deflection disappears and the method reverts to the plain subgradient. This is consistent: deflection accelerates the trajectory far from the optimum, but close to it the local subgradient information is preferable.

3.2.1 Optimal deflection parameter

At each iteration, the deflection parameter can be chosen to minimise the norm of the deflected direction:

$$\gamma^i \in \arg \min_{\gamma \in [0,1]} \|\gamma \mathbf{g}^i + (1 - \gamma) \mathbf{d}^{i-1}\|^2. \quad (3.7)$$

In the Polyak stepsize (3.5) the denominator is $\|\mathbf{d}^i\|^2$; minimising it maximises the step length and hence the progress per iteration. We therefore seek the “most efficient” direction among all convex combinations of $\mathbf{g}^i$ and $\mathbf{d}^{i-1}$. Problem (3.7) is a quadratic in a single variable constrained to $[0, 1]$ and admits a closed-form solution. Define $\varphi(\gamma) = \|\gamma \mathbf{g} + (1 - \gamma) \mathbf{d}^-\|^2$:

$$\varphi(\gamma) = \gamma^2 \|\mathbf{g}\|^2 + 2\gamma(1 - \gamma) \langle \mathbf{g}, \mathbf{d}^- \rangle + (1 - \gamma)^2 \|\mathbf{d}^-\|^2. \quad (3.8)$$

Setting $\varphi'(\gamma) = 0$ gives the unconstrained minimiser

$$\gamma^* = \frac{\|\mathbf{d}^-\|^2 - \langle \mathbf{g}, \mathbf{d}^- \rangle}{\|\mathbf{g} - \mathbf{d}^-\|^2}, \quad (3.9)$$

and the constrained solution is $\gamma^i = \min(1, \max(0, \gamma^*))$.

3.3 The target-level mechanism

The Polyak stepsize in its original form requires the optimal value f^* , which is precisely the quantity we are trying to compute. Recall the **fundamental relationship** for the subgradient method:

$$\|\mathbf{x}^{i+1} - \mathbf{x}^*\|^2 \leq \|\mathbf{x}^i - \mathbf{x}^*\|^2 \underbrace{- 2\alpha^i(f(\mathbf{x}^i) - f^*)}_{\leq 0 \text{ (good)}} + \underbrace{(\alpha^i)^2 \|\mathbf{g}^i\|^2}_{\geq 0 \text{ (bad)}}. \quad (3.10)$$

The choice $\alpha^i = (f(\mathbf{x}^i) - f^*)/\|\mathbf{g}^i\|^2$ maximises the guaranteed reduction in distance to $\mathbf{x}^*$ at each step; however, it requires knowledge of f^* . The **target-level** mechanism circumvents this by maintaining an estimate $f_{\text{lev}} = f_{\text{ref}} - \delta$ of f^* , where:

- f_{ref} is the best function value found so far (the *record value*);
- $\delta > 0$ is an expected improvement margin—the “hope” of how much further we can improve.

If the estimate is accurate ($f_{\text{ref}} - \delta \approx f^*$), the algorithm makes good progress. If the estimate is too optimistic ($f_{\text{ref}} - \delta < f^*$), the Polyak numerator $f(\mathbf{x}^i) - f_{\text{lev}}$ becomes very small or negative, producing tiny or ill-defined steps and stalling the algorithm. When this is detected—after a prescribed amount of travel without significant improvement— δ is reduced (multiplied by $\rho < 1$) and the algorithm retries with a less ambitious target.

3.3.1 The complete algorithm

The full Deflected Subgradient Method with target level is given in Algorithm 2.

Algorithm 2 Deflected Subgradient with Target Level (SGPTL)

Require: f , $\mathbf{w}_0$, $i_{\max}$, $\beta \in (0, 2)$, $\delta_0 > 0$, $R > 0$, $\rho \in (0, 1)$

```

1:  $r \leftarrow 0$ ;  $\delta \leftarrow \delta_0$ ;  $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$ ;  $\mathbf{d}_{-1} \leftarrow \mathbf{0}$ 
2: for  $i = 0, 1, \dots, i_{\max}$  do
3:   Compute  $\mathbf{g} \in \partial f(\mathbf{w}_i)$  (any subgradient)
4:   Compute  $\gamma_i$  via (3.7) (optimal deflection)
5:    $\mathbf{d}_i \leftarrow \gamma_i \mathbf{g} + (1 - \gamma_i) \mathbf{d}_{i-1}$  (deflected direction)
6:    $\alpha_i \leftarrow \beta (f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)) / \|\mathbf{d}_i\|^2$  (Polyak with target level)
7:    $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i \mathbf{d}_i$  (step)
8:    $\bar{f} \leftarrow \min(\bar{f}, f(\mathbf{w}_{i+1}))$  (update record before testing improvement)
9:   if  $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$  then (significant improvement)
10:     $f_{\text{ref}} \leftarrow \bar{f}$ ;  $r \leftarrow 0$ 
11:   else if  $r > R$  then (stalled: reduce target)
12:     $\delta \leftarrow \delta \rho$ ;  $r \leftarrow 0$ 
13:   else
14:     $r \leftarrow r + \alpha_i \|\mathbf{d}_i\|$  (accumulate travelled distance)
15:   end if
16: end for
17: return  $\mathbf{w}_{\text{best}}$  (the iterate achieving  $\bar{f}$ )
```

Remark 3.1 (Pseudocode correction). In a naive presentation, the record update $\bar{f} \leftarrow \min(\bar{f}, f(\mathbf{w}_{i+1}))$ might appear at the end of the loop body, after the if–elseif–else block. This is a logical error: the “significant improvement” branch assigns $f_{\text{ref}} \leftarrow \bar{f}$, which should reflect the *current* best value including $\mathbf{w}_{i+1}$. Placing the record update before the conditional ensures that f_{ref} is correctly set to $\min(\bar{f}_{\text{old}}, f(\mathbf{w}_{i+1}))$ when progress is detected.

The algorithm maintains three key quantities (we write $\bar{f}$ in the pseudocode; in the analysis below, $\bar{f}^i$ denotes the record value at iteration i):

- $\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$: the **record value**—the best f encountered up to iteration i . Since the method is non-monotone, this is the quantity that converges and constitutes the effective output.

- $f_{\text{ref}} - \delta$: the **target level**—the current estimate of f^* used in the Polyak stepsize.
- r : a counter of distance travelled without significant improvement. When r exceeds the patience threshold R , the target is deemed too optimistic and δ is reduced.

3.3.2 Parameters

Remark 3.2 (Parameter roles). The algorithm involves several parameters whose calibration is critical for performance:

- δ_0 (*initial gap estimate*): an estimate of $f(\mathbf{w}_0) - f^*$. If set too large, the target $f_{\text{ref}} - \delta$ is unrealistically low, producing oversized steps and instability; if too small, the steps are insufficient to make meaningful progress. A reasonable choice is $\delta_0 = c \cdot f(\mathbf{w}_0)$ with $c \in [0.01, 0.5]$.
- $\rho \in (0, 1)$ (*target reduction rate*): controls how quickly δ is reduced when progress stalls. Typical values are $\rho = 0.95$ (slow, more iterations but tighter target tracking) or $\rho = 0.9$ (more aggressive reduction).
- $R > 0$ (*patience threshold*): the distance travelled without improvement before δ is reduced. Too small causes premature reductions (overly conservative target); too large delays adaptation.
- $\beta \in (0, 2)$ (*step modulation*): scales the Polyak step. The natural choice is $\beta = 1$. Values in $(1, 2)$ yield more aggressive (but less stable) steps; values in $(0, 1)$ are more conservative.

3.4 Application to our problem

3.4.1 Subgradient computation

To apply the SGPTL algorithm to our LASSO problem

$$f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad (3.11)$$

we need to compute a subgradient $\tilde{\mathbf{g}} \in \partial f(\mathbf{w})$ at each iteration. Decompose $f = q + h$, where $q(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ is the smooth part and $h(\mathbf{w}) = \lambda \|\mathbf{w}\|_1$ is the nonsmooth part. Since both are convex, the sum rule for subdifferentials gives

$$\partial f(\mathbf{w}) = \nabla q(\mathbf{w}) + \lambda \partial \|\mathbf{w}\|_1,$$

where $\nabla q(\mathbf{w}) = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ exists everywhere. The subdifferential of the ℓ_1 norm is given component-wise (see Section 1.5.3). A subgradient of f at $\mathbf{w}$ is therefore

$$\tilde{\mathbf{g}} = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{s}, \quad s_i \in \partial |w_i|, \quad (3.12)$$

where $s_i = \text{sign}(w_i)$ if $w_i \neq 0$ and $s_i = 0$ if $w_i = 0$ (minimum-norm element of the subdifferential, yielding the steepest-descent direction). In Algorithm 2

and throughout this chapter, $\mathbf{g}^i$ denotes the subgradient $\bar{\mathbf{g}}$ evaluated at the i -th iterate $\mathbf{w}_i$.

3.4.2 Convergence

Theorem 3.3 (Convergence of the Deflected Subgradient Method). *Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be convex and let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2 with the convergence conditions satisfied (stepsize-restricted with $\beta^i \leq \gamma^i$, or deflection-restricted). Assume that the subgradients are uniformly bounded: $\|\mathbf{g}^i\| \leq L$ for all i . Then the sequence of record values*

$$\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$$

converges to the optimal value: $\bar{f}^i \rightarrow f^$ as $i \rightarrow \infty$. The worst-case complexity is $O(1/\varepsilon^2)$: to achieve $\bar{f}^i - f^* \leq \varepsilon$, at most $O(L^2/\varepsilon^2)$ iterations are required.*

Proof sketch. The argument builds on the fundamental relationship (3.10), adapted to the deflected case. For the plain subgradient $\mathbf{x}^{i+1} = \mathbf{x}^i - \alpha^i \mathbf{g}^i$, expanding the squared distance gives

$$\begin{aligned} \|\mathbf{x}^{i+1} - \mathbf{x}^*\|^2 &= \|\mathbf{x}^i - \mathbf{x}^*\|^2 - 2\alpha^i \langle \mathbf{g}^i, \mathbf{x}^i - \mathbf{x}^* \rangle + (\alpha^i)^2 \|\mathbf{g}^i\|^2 \\ &\leq \|\mathbf{x}^i - \mathbf{x}^*\|^2 - 2\alpha^i (f(\mathbf{x}^i) - f^*) + (\alpha^i)^2 \|\mathbf{g}^i\|^2, \end{aligned} \quad (3.13)$$

where the inequality uses the subgradient inequality $f(\mathbf{x}^*) \geq f(\mathbf{x}^i) + \langle \mathbf{g}^i, \mathbf{x}^* - \mathbf{x}^i \rangle$. In the deflected case, $\mathbf{d}^i$ replaces $\mathbf{g}^i$ in (3.13). The convergence conditions ensure that an analogous relationship holds for the deflected direction. Under the Polyak stepsize with $\beta^i \leq \gamma^i$:

1. The record value $\bar{f}^i$ is non-increasing by construction.
2. Telescoping (3.13) over $i = 1, \dots, k$ and using $\|\mathbf{x}^{k+1} - \mathbf{x}^*\|^2 \geq 0$ yields

$$\bar{f}^k - f^* \leq \frac{L \|\mathbf{x}^1 - \mathbf{x}^*\|}{\sqrt{k}}, \quad (3.14)$$

which gives $k \geq L^2 \|\mathbf{x}^1 - \mathbf{x}^*\|^2 / \varepsilon^2 = O(1/\varepsilon^2)$. $\square$

Remark 3.4 (Key observations on convergence).

- **Non-monotonicity.** It is the sequence of record values $\bar{f}^i$ that converges, *not* the sequence of iterates $\mathbf{w}_i$ or function values $f(\mathbf{w}_i)$. The method is intrinsically non-monotone: $f(\mathbf{w}_{i+1}) > f(\mathbf{w}_i)$ can and does happen frequently. For this reason the algorithm returns the best iterate found, not the last one.
- **Optimality of the rate.** The $O(1/\varepsilon^2)$ complexity is identical to that of the plain subgradient method. This is an intrinsic limitation of nonsmooth optimisation: for convex, L -Lipschitz functions that are not C^1 , the lower bound is $\Omega(L^2/\varepsilon^2)$, so the method is worst-case optimal. Deflection does not improve the worst-case complexity, but significantly improves practical behaviour by reducing oscillations.

- **Comparison with smooth methods.** For smooth, strongly convex functions the complexity is $O(\log(1/\varepsilon))$. Concretely: going from $\varepsilon = 10^{-3}$ to $\varepsilon = 10^{-4}$ with a subgradient method requires $(10^{-3}/10^{-4})^2 = 100\times$ more iterations.

Verification of hypotheses for our problem. We check that the assumptions of Theorem 3.3 are satisfied by (3.11):

1. **Convexity.** $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$ is convex, being the sum of a convex smooth function (q , with Hessian $2\mathbf{X}^\top \mathbf{X} \succeq 0$) and a convex norm. If $\mathbf{X}$ has full column rank, q is τ -strongly convex with $\tau = 2\lambda_{\min}(\mathbf{X}^\top \mathbf{X}) > 0$, so f is also strongly convex and the minimum is unique. In this case the theoretical lower bound for nonsmooth optimisation improves from $\Omega(1/\varepsilon^2)$ to $\Omega(1/\varepsilon)$. However, the DSM as implemented does not exploit strong convexity, so its worst-case rate remains $O(1/\varepsilon^2)$.
2. **Bounded subgradients.** The ℓ_1 component $\lambda \mathbf{s}$ has norm at most $\lambda\sqrt{H}$ (since $|s_i| \leq 1$). The smooth component $2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows linearly with $\|\mathbf{w}\|$, so f is not globally Lipschitz on $\mathbb{R}^H$. However, the target-level mechanism prevents arbitrarily large steps, so the iterates remain confined to a bounded set and $\|\mathbf{g}^i\|$ is uniformly bounded along the generated sequence.

3.4.3 Expected properties on the ELM problem

Based on the theoretical analysis, the following behaviour is expected when applying DSM to the ELM LASSO problem (3.11):

- **Non-monotone convergence.** The function value $f(\mathbf{w}_i)$ oscillates throughout the run. The quantity to monitor is the record value $\bar{f}^i$, which decreases monotonically by construction. In convergence plots it is essential to display $\bar{f}^i$ rather than $f(\mathbf{w}_i)$.
- **Sublinear convergence.** In a semilogarithmic plot ($\log(\bar{f}^i - f^*)$ vs. iterations), the curve flattens progressively: initial improvements are rapid, but progress slows at each decade of precision. This contrasts with the typically linear convergence of IRLS, which produces a straight line on the same scale. The reason is the $O(1/\varepsilon^2)$ complexity: doubling the desired precision requires roughly quadrupling the number of iterations.
- **Difficulty at high accuracy.** Achieving gaps $\bar{f}^i - f^* < 10^{-4}$ requires a prohibitive number of iterations due to the $O(1/\varepsilon^2)$ rate. This is a theoretical limitation of the approach, not an implementation defect.
- **Low per-iteration cost.** Each iteration costs $O(MH)$, dominated by two matrix-vector products with $\mathbf{X} \in \mathbb{R}^{M \times H}$. This makes the method competitive in CPU time when low accuracy suffices, particularly for large H where the $O(H^3)$ cost of IRLS's Cholesky factorisation becomes prohibitive.

- **Parameter sensitivity.** Performance depends critically on the calibration of δ_0 , ρ , R , and β . Poor choices may lead to extremely slow convergence or instability. A preliminary tuning phase is recommended.
- **No reliable stopping criterion.** Unlike smooth methods where $\|\nabla f(\mathbf{w}^*)\| = 0$ provides a natural stopping criterion, in the nonsmooth case $0 \in \partial f(\mathbf{w}^*)$ but other elements of $\partial f(\mathbf{w}^*)$ can have large norm, so $\|\mathbf{g}^i\|$ can remain large even at the optimum. The only practical criterion is to monitor $\bar{f}^i$ and stop after a prescribed iteration budget or when $\bar{f}^i$ ceases to improve appreciably.
- **Approximate sparsity.** Unlike IRLS, which tends to produce components that are exactly zero thanks to the weighted least-squares structure, the DSM yields solutions where “small” components are close to zero but rarely exactly so. A post-processing thresholding step may be needed to enforce exact sparsity below a chosen tolerance.
- **Effect of λ .** Larger λ produces a larger subgradient component from the ℓ_1 term, which increases oscillations and may slow convergence. However, the target-level mechanism adapts to this automatically. The sparsity of the solution increases with λ , as the ℓ_1 penalty pushes more weights toward zero.

Chapter 4

Algorithm Comparison

Both IRLS (Chapter 2) and the Deflected Subgradient Method (Chapter 3) solve the same LASSO problem

$$\min_{\mathbf{w} \in \mathbb{R}^H} f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1, \quad (4.1)$$

yet they differ fundamentally in *how* they handle the non-smoothness of the ℓ_1 penalty and in the trade-offs that result. This chapter compares the two methods along the dimensions that matter most in practice: mathematical strategy, convergence behaviour, computational cost, solution quality, and suitability for the ELM setting.

4.1 Core strategy: how each algorithm handles non-smoothness

The root difficulty in (4.1) is the ℓ_1 term: it is convex but non-differentiable wherever any $w_i = 0$, so classical gradient methods cannot be applied directly. The two algorithms adopt opposite philosophies.

IRLS — smoothing via majorization. IRLS sidesteps the non-smoothness by never minimising f directly. At each iteration, it constructs a smooth *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ that majorizes f (Section 2.3):

$$Q(\mathbf{w}, \mathbf{w}_k) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + 2\lambda \mathbf{w}^T \mathbf{W}_k \mathbf{w},$$

where the non-smooth $\|\mathbf{w}\|_1$ has been replaced by the quadratic $\mathbf{w}^T \mathbf{W}_k \mathbf{w}$, with weights $(\mathbf{W}_k)_{ii} = \max(|w_{k,i}|, \varepsilon_{\text{thr}})^{-1/2}$ that encode the current estimate of the solution's sparsity pattern (Section 2.1). Minimising $Q(\cdot, \mathbf{w}_k)$ amounts to solving a weighted L_2 -regularised least-squares problem with a closed-form solution via the normal equations (Section 2.2).

DSM — direct subgradient with memory. DSM confronts the non-smoothness head-on. It descends along a *subgradient* $\tilde{\mathbf{g}} \in \partial f(\mathbf{w}_i)$ at every step, computing it exactly via the sum rule $\tilde{\mathbf{g}} = 2\mathbf{X}^T(\mathbf{X}\mathbf{w}_i - \mathbf{y}) + \lambda\mathbf{s}$ (Section 3.4.1). To combat the zig-zag oscillations of the plain subgradient method (Section 3.1), the direction is *deflected*: it blends the current subgradient with the previous direction (Section 3.1.2), and the Polyak stepsize is controlled via a target-level estimate of f^* (Section 3.3).

Key structural difference. IRLS transforms a non-smooth problem into a sequence of smooth ones, then solves each exactly (via a linear system solve). DSM never smooths the objective; instead it relies on the convergence theory for subgradient methods, accepting that individual steps may increase f .

4.2 Convergence behaviour

The convergence properties of the two algorithms differ in both *rate* and *monotonicity*.

4.2.1 Monotonicity

IRLS is **monotone**: the MM framework guarantees $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ at every iteration (Section 2.3). This makes convergence easy to monitor and diagnose — a non-decreasing plot of $f(\mathbf{w}_k)$ immediately signals a bug.

DSM is **non-monotone**: individual function values $f(\mathbf{w}_i)$ oscillate throughout the run (Section 3.4.2). The meaningful quantity is the *record value* $\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$, which decreases by construction but may improve only sporadically. Plotting $f(\mathbf{w}_i)$ without tracking $\bar{f}^i$ gives a misleading picture.

4.2.2 Rate

IRLS converges at a **linear** (geometric) rate: on a semilogarithmic plot of $f(\mathbf{w}_k) - f^*$ versus iteration number, the curve is approximately a straight line (Section 2.5). In practice, 10–50 iterations suffice to reach high accuracy.

DSM converges at a **sublinear** rate, $O(1/\varepsilon^2)$ (Theorem 3.3): on the same semilogarithmic plot, the curve flattens progressively. This rate is information-theoretically optimal for nonsmooth convex functions (Section 3.4.2), so the gap is intrinsic to the approach, not an implementation defect. Concretely, achieving $\varepsilon = 10^{-4}$ requires roughly $100\times$ more iterations than $\varepsilon = 10^{-3}$.

4.2.3 Convergence on the ELM problem

For the ELM LASSO problem, $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1)$ has full column rank with high probability (since $\mathbf{W}_1$ is random and σ is a nonlinear activation), so f is strictly convex and the unique minimiser $\mathbf{w}^*$ exists. This means IRLS converges to $\mathbf{w}^*$ (Theorem 2.2) with a linear rate governed by the condition number of the

weighted normal-equation matrices. DSM also converges to $\mathbf{w}^*$ (Theorem 3.3), and the strong convexity of f improves the theoretical lower bound from $O(1/\varepsilon^2)$ to $O(1/\varepsilon)$, though the algorithm does not exploit this structure.

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	IRLS	DSM
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(kH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

IRLS requires solving an $H \times H$ symmetric positive definite linear system at every iteration (Section 2.2). With Cholesky factorisation the cost is $O(H^3/3)$ per iteration. With k steps of Conjugate Gradient it is $O(kH^2)$, preferable when H is large and $\mathbf{Q}_k$ is well-conditioned. The matrices $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T \mathbf{y}$ are pre-computed once at cost $O(MH^2)$ and $O(MH)$ respectively.

DSM requires only two matrix-vector products per iteration (to compute the subgradient: $\mathbf{X}\mathbf{w}_i$ costs $O(MH)$ and $\mathbf{X}^T(\cdot)$ costs $O(MH)$), giving a total per-iteration cost of $O(MH)$ (Section 3.4.1). No precomputation of $\mathbf{X}^T \mathbf{X}$ is needed.

4.3.2 Total cost

Let k_{IRLS} and k_{DSM} denote the respective iteration counts.

$$\begin{aligned} \text{IRLS total: } & O(MH^2) + k_{\text{IRLS}} \cdot O(H^3), \\ \text{DSM total: } & k_{\text{DSM}} \cdot O(MH). \end{aligned}$$

Since IRLS converges linearly and DSM converges sublinearly, the iteration counts satisfy $k_{\text{IRLS}} \ll k_{\text{DSM}}$ for a given accuracy. The regime where DSM is competitive is therefore when H is large enough that $H^3 \gg MH$, i.e. $H^2 \gg M$, so that the Cholesky cost of IRLS dominates.

For the ELM problem, M is the number of training samples and H is the hidden layer size. Typical configurations have $M \gg H$ (many samples, moderate hidden layer), so $O(MH^2) \gg O(H^3)$ and the pre-computation of $\mathbf{A}$ dominates IRLS's total cost. In this regime, the total costs are roughly $O(MH^2)$ for IRLS and $O(k_{\text{DSM}} \cdot MH)$ for DSM — IRLS wins whenever $H \lesssim k_{\text{DSM}}$, which holds for any reasonable accuracy requirement.

4.4 Solution quality

4.4.1 Sparsity

A central goal of LASSO is to produce a *sparse* solution $\mathbf{w}^*$ with many components exactly zero, which in the ELM context corresponds to identifying the most relevant hidden neurons and discarding the rest.

IRLS promotes **exact sparsity**: as iterations proceed, components $w_{k,i}$ with small magnitude receive increasingly large weights $(\mathbf{W}_k)_{ii}$, pushing them further toward zero. At convergence, many components are exactly zero (within ε_{thr}), yielding a genuinely sparse weight vector (Section 2.7). The sparsity structure emerges naturally from the algorithm’s geometry.

DSM produces **approximate sparsity**: the iterates drift near zero for irrelevant components, but they are rarely driven to exact zero because the subgradient steps carry no mechanism that enforces zeros. A post-processing thresholding step (setting $w_i \leftarrow 0$ if $|w_i| < \tau$ for some small τ) is typically needed to obtain a truly sparse solution (Section 3.4.3).

4.4.2 Accuracy

For a fixed computational budget (number of iterations or wall-clock time), IRLS reaches substantially higher accuracy than DSM on small-to-moderate problems, due to its linear convergence rate. DSM can reach competitive accuracy when the per-iteration cost advantage of $O(MH)$ versus $O(H^3)$ outweighs the slower convergence rate.

4.5 Parameter sensitivity

IRLS. The algorithm has two main hyperparameters: the regularisation strength λ (shared with the problem formulation) and the safety threshold ε_{thr} . The latter controls the trade-off between numerical stability and approximation quality: too large degrades sparsity, too small risks division by zero. A value around 10^{-8} is a robust default. The convergence stopping criterion $\varepsilon_{\text{stop}}$ is straightforward to tune: the relative change $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| / \max(1, \|\mathbf{w}_k\|)$ provides a scale-invariant and interpretable criterion (Section 2.4).

DSM. The algorithm involves four additional hyperparameters: δ_0 , ρ , R , and β (Section 3.3.2). Their calibration is critical and interdependent: a poor choice of δ_0 dominoes into poor target levels, which cause either oversized steps (δ_0 too large) or stalling (δ_0 too small). The patience threshold R and reduction rate ρ control adaptivity. Unlike IRLS, there is no reliable stopping criterion based on iterate change, since the subgradient norm may remain large even at the optimum (Section 3.4.3). A fixed iteration budget is the most practical approach.

4.6 Summary and guidelines for the ELM problem

Criterion	IRLS	DSM
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(1/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	ε_{thr}	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H , moderate accuracy

When to prefer IRLS. IRLS is the method of choice when:

- High solution accuracy is required (e.g. for precise feature selection or reproducibility of sparsity patterns).
- H is small to moderate relative to M , so the $O(H^3)$ per-iteration cost is affordable.
- Exact zeros in $\mathbf{w}^*$ are important (e.g. to identify which hidden neurons to prune).
- Easy debugging is preferred: the monotone decrease of f is a strong sanity check.

When to prefer DSM. DSM is competitive when:

- H is large enough that Cholesky factorisation is infeasible or too slow ($H^2 \gg M$).
- Moderate accuracy ($\varepsilon \approx 10^{-3}$) suffices.
- Memory is a constraint: DSM does not require storing $\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{H \times H}$, only the current iterate and direction.
- The problem changes online (e.g. streaming data), where recomputing $\mathbf{A}$ is expensive.

Practical verdict for the ELM problem. In the typical ELM setting (fixed random $\mathbf{W}_1$, offline training), M is large and H is chosen much smaller, so $M \gg H$. In this regime IRLS is clearly preferable: the precomputed $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ is computed once at cost $O(MH^2)$, and each subsequent iteration costs only $O(H^3)$, which is modest for typical $H \leq 1000$. The monotone convergence, reliable stopping criterion, exact sparsity, and minimal parameter tuning all favour IRLS. DSM serves as a useful baseline and as a fallback for very large H , where it trades accuracy for scalability.